

COMP 356
Programming Language Structures
Notes for Chapter 11 of *Concepts of Programming Languages*
Data Abstraction and Encapsulation

Abstraction:

- (dictionary) the act of taking away or separating
- suppressing unnecessary detail to allow a higher level of understanding

Kinds of abstraction in programming:

- **process abstraction**
 - replacing a computation with a call to a subprogram that performs the computation
 - allows readers to understand a program without keeping in mind every detail of how each subprogram is implemented
 - “client” code can use a subprogram without knowing how it is implemented
- **data abstraction**
 - hiding the representation of data and the implementation of operations on that data
 - along with proper documentation, allows client code to use a data type without knowing how it is implemented

Abstraction is the key to managing software complexity. Human beings can NOT develop and understand large software systems one line of code at a time.

1 Abstract Data Types

An **abstract data type** (ADT) is a data type that satisfies the following conditions:

- the declarations of the type and the operations on objects of the type are contained in the same syntactic unit (**encapsulation**). (The implementation of the operations may be in a different syntactic unit.)
- the representation of the type is hidden from client code
- client code does not need to know how operations on objects of the type are implemented
- client code can declare variables of the ADT and create instances of the ADT

Advantages of abstract data types:

- logical organization of programs
- separate compilation
- ability to change the representation without breaking client code
- increased reliability - clients can not make arbitrary changes to objects of the abstract data type, so data **invariants** can be maintained

2 Language Approaches to Supporting ADTs

- classes
- packages

2.1 Packages

Packages in Ada provide:

- encapsulation of data and operations
- hiding of data and operations from client code (private)
- the ability to define 0, 1 or many ADTs in one package

An Ada package can have two parts - the **specification** and the **body**. The package specification provides:

- the names of exported types (types visible outside of the package)
- the signatures of subprograms (functions and procedures) visible outside of the package
- the representation of the exported types (in a **private** section)

The package specification is intended to be visible to client code.

The package body provides the implementations of the subprograms, along with any “helper” subprograms not visible outside of the package. The package body is NOT intended to be visible to client code.

Notes:

- a package can export multiple multiple types with associated operations
- no support for constructors (or destructors) is provided

2.2 Classes

Key features:

- encapsulation of data and operations
- hiding of data and operations from client code (private)
- define a single type (each class defines one ADT)
- implicit calling of constructors and destructors

2.2.1 Classes in C++

Terminology:

- **data members** - instance variables
- **member functions** - methods
- **members** - data members and member functions
- **destructor** - an operation that is implicitly called when the lifetime of an object ends. Often used to deallocate any heap-dynamic data members.

Other notes:

- access modifiers (**public**, **protected**, **private**) refer to clauses (sections of classes), not individual members
- instances of a C++ class can be stack-dynamic or heap-dynamic. Heap-dynamic instances are referred to by a pointer.
- member functions can be defined directly within the class (inline), or in another implementation file. Common style:

- define the class within a **header** (.h) file. For most member functions, only the prototype (signature) will appear in the header file.
- implement methods in an implementation (.C) file, using the **scope resolution** operator (::) to indicate the class they belong to.

Example - points and line segments in the real plane:

File point.h:

```
#include <iostream>
#include <math.h>

class Point { // points in the real plane
private:
    double x;
    double y;

public:
    Point() { // default constructor
        x = 0;
        y = 0;
    }

    Point(double xv, double yv) {
        x = xv;
        y = yv;
    }

    virtual void print() {
        std::cout << "(" << x << ", " << y << ")";
    }

    virtual double getX() { return x;}
    virtual double getY() { return y;}
};

class LineSegment { // line segments in the real plane
private:
    Point *start;
    Point *end;

public:
    LineSegment() {
        start = NULL;
        end = NULL;
    }

    LineSegment(double x1, double y1, double x2, double y2) {
        start = new Point(x1, y1);
        end = new Point(x2, y2);
    }

    ~LineSegment() { // destructor used to deallocate the points
        delete start;
        delete end;
    }
}
```

```

    virtual void print() {
        start->print();
        std::cout << " to ";
        end->print();
    }

    virtual double length();
    // length of line segment
};

```

File: point.C

```

#include "point.h"

double square(double x) {return x * x;}

double LineSegment::length() {
    // length of line segment
    return sqrt(square(start->getX() - end->getX()) +
                square(start->getY() - end->getY()));
}

```

File: pointTest.C

```

#include <iostream>
#include "point.h"

int main() {
    LineSegment *ls = new LineSegment(0, 0, 10.5, 2.3); // heap-dynamic instance
    LineSegment ls2(0, 0, 5.8, 10.3); // stack-dynamic instance
    ls->print();
    std::cout << std::endl;
    ls2.print();
    std::cout << std::endl;
    std::cout << ls->length() << std::endl;
    delete ls;
}

```

3 Parameterized ADTs

ADTs in some languages (Ada, C++) can take parameters, and these parameters can be values or types. For example, a C++ **template** stack class could take as parameters:

- the maximum size of a stack (value)
- the type of element stored in the stack (type)

When an instance of a template class is created, the parameters must be supplied as part of the type.

File: stack.C

```

#include <iostream>

template <int size, class ElemType> class Stack {
private:
    ElemType *stackPtr;

```

```

    int topElem;
    // invariant: -1 <= topElem <= size - 1

public:
    Stack() {
        stackPtr = new ElemType[size];
        topElem = -1; // no elements in stack yet
    }

    ~Stack() {
        delete stackPtr;
    }

    void push(ElemType e) {
        if (topElem >= size - 1) {
            std::cout << "Error: stack is full." << std::endl;
        } else {
            topElem++;
            stackPtr[topElem] = e;
        }
    }

    void pop() {
        if (topElem == -1) {
            std::cout << "Error: stack is empty." << std::endl;
        } else {
            topElem--;
        }
    }

    ElemType top() {
        // pre: topElem > -1
        return stackPtr[topElem];
    }
};

int main() {
    Stack<50, int> s; // invokes default constructor
    s.push(3);
    s.push(4);
    s.push(5);
    std::cout << s.top() << std::endl;
    s.pop();
    std::cout << s.top() << std::endl;

    Stack<20, double> *s2 = new Stack<20,double>();
    s2->push(3.4);
    s2->push(5.4);
    s2->pop();
    std::cout << s2->top() << std::endl;
    delete s2;
}

```

Java 5.0 generic classes are very similar to C++ template classes. The following generic stack class differs from the C++ version in only three significant ways:

- the type parameter must be a class type (generic classes can not take primitive types as parameters)
- the size of the array can not be passed as part of instantiating a template
- programs can not create arrays with a generic base type

The example avoids this last problem by using an array of base type `Object`.

```
public class Stack<ElemType> {
    private Object [] theStack; // can't create a generic array
    private int topElem;
    // invariant: -1 <= topElem <= theStack.length - 1

    public Stack(int size) {
        // pre: size > 0
        theStack = new Object [size];
        topElem = -1;
    }

    void push(ElemType e) {
        if (topElem >= theStack.length - 1) {
            System.out.println("Error: stack is full.");
        } else {
            topElem++;
            theStack[topElem] = e;
        }
    }

    void pop() {
        if (topElem == -1) {
            System.out.println("Error: stack is empty.");
        } else {
            topElem--;
        }
    }

    ElemType top() {
        // pre: topElem > -1
        return (ElemType) theStack[topElem];
    }

    public static void main(String argv []) {
        Stack<String> s = new Stack<String>(50); // invokes default constructor
        s.push("antelope");
        s.push("zebra");
        s.push("gazelle");
        System.out.println("Top element: " + s.top());
        s.pop();
        System.out.println("Top element after pop: " + s.top());

        Stack<Integer> s2 = new Stack<Integer>(20);
        s2.push(new Integer(3));
        s2.push(new Integer(4));
        s2.push(new Integer(5));
        System.out.println("Top element: " + s2.top());
    }
}
```

```

        s2.pop();
        System.out.println("Top element after pop: " + s2.top());
    }
}

```

The `LinkedList` class on the course web page shows how to implement a generic `Stack` class with a linked list and a nested class.

Generic classes can use the same wildcards for type parameters as generic methods. For example, the following prints a `Collection` that has been instantiated with any type of object.

```

public static void printCollection(Collection<?> col) {
    Iterator<?> it = col.iterator();
    while (it.hasNext()) {
        System.out.println(it.next());
    }
}

```

Note that if the signature for this method were:

```

public static void printCollection(Collection<Object> col) {

```

it could only be called with a collection of instances of class `Object` - no subclass instances could be used.

4 Encapsulation Mechanisms

Idea: large programs need more structure than just ADTs - there should be some way to organize logically related ADTs into “units”.

4.1 Packages in Java

In Java, a package is a collection of classes. Each source file for such a class begins with:

```

package packageName;

```

where *packageName* is the name of the package.

Any member of a Java class (instance variable, method) declared without an access modifier (such as `public` or `private`) has package access - it is visible to all other classes in the same package. This is often useful for closely related classes - if class `LinkedList` and class `ListNode` are both defined in package `MyList`, then class `LinkedList` can use the instance variables of `ListNode` directly (if they are declared with package access), but code outside of the package can not.

Java packages are also useful for avoiding name clashes. Classes declared in a package are referred to using the package name from outside of the package, i.e.:

```

MyList.LinkedList L = new MyList.LinkedList();

```

Another package can define a class called `LinkedList`, and the two classes can be distinguished. If there are no name clashes, then a package can be imported and names defined in the package can be used without the package name qualifier, i.e.:

```

import MyList.LinkedList;
...

```

```

LinkedList L = new LinkedList();

```

Notes:

- a package must be defined in a directory with the same name as the package

- packages can be nested:
 - the child package is placed in a subdirectory of the parent package directory
 - a child package is specified as parent package name . child package name, i.e. `AllLists.MyList.LinkedList` if `MyList` is a child package of `AllLists`
 - child packages have no special access to parent packages (and vice versa) - package nesting is only an organizational mechanism
- the **classpath** specifies which directories java and javac search for packages and classes

4.2 Packages in Ada

Ada packages are very similar to Java packages:

- packages can be nested
- packages provide access control

4.3 Namespaces in C++

Namespaces in C++ provide only name management (no access control). For example:

```
namespace MyList {
    class ListNode {
        // define class here
    };

    class LinkedList {
        // define class here
    };
}
```

defines classes `ListNode` and `LinkedList` in namespace `MyList`. Code outside of the namespace can refer to names defined inside the namespace using scope resolution, i.e.:

```
MyList::LinkedList *l = new MyList::LinkedList;
```

so a large program can have multiple classes called `LinkedList`.

Notes:

- one namespace can be split across multiple files just by using another **namespace** block (with the same name)
- code within a namespace does not need to qualify the names of other classes, functions etc. defined in the same namespace
- a namespace can be imported with keyword **using**, i.e. `using namespace MyList;`

To get the effect of package access in C++, programmers can use **friend** functions and classes. For example:

```
class ListNode {
    friend class LinkedList;
    ...
};
```

Now, class `LinkedList` can access the private members of class `ListNode`, but no other class can. Note that namespaces and friends are orthogonal concepts.

4.4 Assemblies in C#

C# assemblies:

- are somewhat similar to Java packages
- are collections of several files
- appear to application programs as a single executable or DLL

The **internal** modifier makes class members visible to all classes in the same assembly (like package access in Java).

C# also includes C++ style namespaces.